EXVUL

SMART CONTRACT AUDIT REPORT

Bullion Pool Smart Contract

AUGUST 2026

Contents

1. EXECUTIVE SUMMARY **4**
1.1 Methodology 4

2. FINDINGS OVERVIEW **7**
2.1 Project Info And Contract Address 7
2.2 Summary 8
2.3 Key Findings 9

3. DETAILED DESCRIPTION OF FINDINGS **10**
3.1 Inflated matchedUsd accounting allows recycled deposits to distort fee-share tiers . . . 10
3.2 Mutable prediction inputs can mislead users into prefunding an unrecoverable address . 13
3.3 Spot-price accounting can inflate fee-share tiers for pool-priced deposits 16
3.4 Silent balanceOf fallback can misaccount collected fees and principal 20
3.5 positionOriginalUsd is recorded from desired mint amounts instead of actual deposited amounts 22
3.6 Stale operator privileges survive admin rotation and preserve execution control 25
3.7 Nominal deposit amounts can overstate fee-tier accounting for supported non-standard tokens 27
3.8 Unbounded fee-share bps can block fee collection and position closure 30
3.9 Manager external updates can make existing ContrAccount instances incompatible with managed operations 32
3.10 ContrAccount admin is stored but never used 34
3.11 zapCreatePositionETH V1 does not ensure swap.amountIn is bounded by ethAmount . 36

4. CONCLUSION **39**

5. APPENDIX **40**
5.1 Basic Coding Assessment 40
5.1.1 Apply Verification Control 40
5.1.2 Authorization Access Control 40
5.1.3 Forged Transfer Vulnerability 40
5.1.4 Transaction Rollback Attack 41
5.1.5 Transaction Block Stuffing Attack 41
5.1.6 Soft Fail Attack Assessment 41
5.1.7 Hard Fail Attack Assessment 42
5.1.8 Abnormal Memo Assessment 42
5.1.9 Abnormal Resource Consumption 42

5.1.10 Random Number Security 43
5.2 Advanced Code Scrutiny 43
5.2.1 Cryptography Security 43
5.2.2 Account Permission Control 43
5.2.3 Malicious Code Behavior 44
5.2.4 Sensitive Information Disclosure 44
5.2.5 System API 44

6. DISCLAIMER **45**

7. REFERENCES **46**

8. About Exvul Security **47**

1. EXECUTIVE SUMMARY

ExVul Web3 Security was engaged by **Bullion Pool** to review smart contract implementation. The assessment was conducted in accordance with our systematic approach to evaluate potential security issues based upon customer requirement. The report provides detailed recommendations to resolve the issue and provide additional suggestions or recommendations for improvement.

The outcome of the assessment outlined in chapter 3 provides the system's owners a full description of the vulnerabilities identified, the associated risk rating for each vulnerability, and detailed recommendations that will resolve the underlying technical issue.

1.1 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [10] which is the gold standard in risk assessment using the following risk models:

- **Likelihood**: represents how likely a particular vulnerability is to be uncovered and exploited in the wild.
- **Impact**: measures the technical loss and business damage of a successful attack.
- **Severity**: determine the overall criticality of the risk.

Likelihood can be: High, Medium and Low and impact are categorized into: High, Medium, Low, Informational. Severity is determined by likelihood and impact and can be classified into five categories accordingly: Critical, High, Medium, Low, Informational shown in table 1.1.

Likelihood \ IMPACT	Informational	Low	Medium	High
High	INFO	MEDIUM	HIGH	CRITICAL
Medium	INFO	LOW	MEDIUM	HIGH
Low	INFO	LOW	LOW	MEDIUM

Table 1.1 Overall Risk Severity

To evaluate the risk, we will be going through a list of items, and each would be labelled with a severity category. The audit was performed with a systematic approach guided by a comprehensive assessment list carefully designed to identify known and impactful security issues. If our tool or analysis does not identify any issue, the contract can be considered safe regarding the assessed item. For any discovered issue, we might further deploy contracts on our private test environment and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.2.

- **Basic Coding Bugs**: We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- **Code and business security testing**: We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- **Additional Recommendations**: We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

Category	Assessment Item
Basic Coding Assessment	• Apply Verification Control • Authorization Access Control • Forged Transfer Vulnerability • Forged Transfer Notification • Numeric Overflow • Transaction Rollback Attack • Transaction Block Stuffing Attack • Soft Fail Attack • Hard Fail Attack • Abnormal Memo • Abnormal Resource Consumption • Secure Random Number

Advanced Source Code Scrutiny	• Asset Security • Cryptography Security • Business Logic Review • Source Code Functional Verification • Account Authorization Control • Sensitive Information Disclosure • Circuit Breaker • Blacklist Control • System API Call Analysis • Contract Deployment Consistency Check • Abnormal Resource Consumption
Additional Recommendations	• Semantic Consistency Checks • Following Other Best Practices

Table 1.2: The Full List of Assessment Items

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [14], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development.

2. FINDINGS OVERVIEW

2.1 Project Info And Contract Address

Project Name	Audit Time	Language
Bullion Pool	07/08/2026 - 15/08/2026	Solidity

Client

合同会社プリンシーデジアート
東京都港区新橋 2-16-1-708
電話番号 03-6770-2411

File	File Hash (MD5)
ContrAccount.sol	2B099D1E53537D10C0404961C39E1E13
Factory.sol	583158BBBA7FF2EFFE4809B2C1D3686B
FeeRule.sol	77703AAC1CA4684FAB4701EAA47C67BE
Manager.sol	08C8222D76A245ABC5140B761B5892F4

2.2 Summary

Severity	Found
CRITICAL	0
HIGH	0
MEDIUM	5
LOW	4
INFO	2



2.3 Key Findings

Severity	Findings Title	Status
MEDIUM	Inflated matchedUsd accounting allows recycled deposits to distort fee-share tiers	Acknowledge
MEDIUM	Mutable prediction inputs can mislead users into prefunding an unrecoverable address	Acknowledge
MEDIUM	Spot-price accounting can inflate fee-share tiers for pool-priced deposits	Acknowledge
MEDIUM	Silent balanceOf fallback can misaccount collected fees and principal	Fixed
MEDIUM	positionOriginalUsd is recorded from desired mint amounts instead of actual deposited amounts	Acknowledge
LOW	Stale operator privileges survive admin rotation and preserve execution control	Fixed
LOW	Nominal deposit amounts can overstate fee-tier accounting for supported non-standard tokens	Acknowledge
LOW	Unbounded fee-share bps can block fee collection and position closure	Acknowledge
LOW	Manager external updates can make existing ContrAccount instances incompatible with managed operations	Fixed
INFO	ContrAccount admin is stored but never used	Fixed
INFO	zapCreatePositionETH V1 does not ensure swap.amountIn is bounded by ethAmount	Acknowledge

Table 2.3: Key Audit Findings

3. DETAILED DESCRIPTION OF FINDINGS

3.1 Inflated matchedUsd accounting allows recycled deposits to distort fee-share tiers

SEVERITY: MEDIUM **STATUS:** Acknowledge

PATH:

ContrAccount.sol

DESCRIPTION:

depositToken() and depositNative() increase matchedUsd immediately after quoting the deposited asset in USD, but none of the owner withdrawal paths reduce the same accounting variable. matchedUsd is only decreased inside closePosition(), and only for owner-initiated closes.

```
    function depositToken(address token, uint256 amount)
        external
        onlyOwner
        nonReentrant
    {
        require(amount > 0, "INVALID_AMOUNT");
        // ... transferFrom(msg.sender, address(this), amount) ...

        emit DepositToken(owner, address(this), token, amount);

        uint256 usdValue =
            IManagerQuote(manager).quoteTokenToUsd(token, amount);

        matchedUsd += usdValue;
        // ...
    }

    function withdrawToken(address token, uint256 amount)
        external
        onlyOwner
        nonReentrant
    {
        (bool ok, bytes memory ret) =
            token.call(
```

```
                abi.encodeWithSelector(
                    IERC20.transfer.selector,
                    owner,
                    amount
                )
            );

        require(ok, 'TRANSFER_CALL_FAILED');

        if (ret.length > 0) {
            require(abi.decode(ret, (bool)), 'TRANSFER_FAILED');
        }

        emit TokenWithdrawn(token, amount);
    }
```

The same asymmetry exists in `withdrawAllToken()`, `withdrawETH()` and `withdrawAllETH()`. Fee distribution derives the user share directly from `feeRule.getUserFeeBps(matchedUsd)`, so the account tiering state is driven entirely by the cumulative deposit total.

```
        uint16 bps = IFeeRule(feeRule).getUserFeeBps(matchedUsd);

        feeAmount0ToOwner = (delta0 * bps) / 10000;
        feeAmount1ToOwner = (delta1 * bps) / 10000;

        protocolFee0 = delta0 - feeAmount0ToOwner;
        protocolFee1 = delta1 - feeAmount1ToOwner;
```

IMPACT:

The same capital can be deposited, withdrawn, and redeposited repeatedly to accumulate an arbitrarily large `matchedUsd` value that no longer reflects capital still committed to the account or LP lifecycle. With the configured rules in `FeeRule.sol` (40% below 3,000 USD, 50% from 3,000 USD, 60% from 30,000 USD), an account holding a small balance can cycle it a few times to reach the top tier and permanently reduce the protocol fee share on all later collections.

RECOMMENDATIONS:

Decrease `matchedUsd` whenever the owner withdraws unmatched assets from the account, so that the accounting variable tracks capital currently committed rather than cumulative deposit volume.

```
 function withdrawToken(address token, uint256 amount) external onlyOwner
    nonReentrant {
     ...
+    uint256 usdValue = IManagerQuote(manager).quoteTokenToUsd(token,
    amount);
+    if (usdValue >= matchedUsd) {
+        matchedUsd = 0;
+    } else {
+        matchedUsd -= usdValue;
+    }
     emit TokenWithdrawn(token, amount);
 }

 function withdrawETH(uint256 amount) external onlyOwner nonReentrant {
     ...
+    uint256 usdValue = IManagerQuote(manager).quoteNativeToUsd(amount);
+    if (usdValue >= matchedUsd) {
+        matchedUsd = 0;
+    } else {
+        matchedUsd -= usdValue;
+    }
     emit ETHWithdrawn(amount);
 }
```

The same adjustment must be applied to `withdrawAllToken()` and `withdrawAllETH()`.

3.2 Mutable prediction inputs can mislead users into prefunding an unrecoverable address

SEVERITY: MEDIUM **STATUS:** Acknowledge

PATH:

Factory.sol

DESCRIPTION:

`predictContrAccount()` presents a deterministic CREATE2 account preview, but the returned address is derived not only from `owner` and `salt`. It also depends on the constructor arguments `manager`, `feeRecipient` and `feeRule`, all of which the admin can change at any time through `setManager()`, `setFeeRecipient()` and `setFeeRule()`.

```
    function createContrAccount() external payable returns (address
    contr) {
        // ...
        bytes32 salt = _computeSalt(msg.sender);

        contr = address(new ContrAccount{salt: salt}(
            msg.sender,
            manager,
            positionManager,
            swapRouter,
            feeRecipient,
            feeRule
        ));
        // ...
    }

    function predictContrAccount(address owner) external view returns
    (address predicted) {
        bytes32 salt = _computeSalt(owner);

        bytes memory bytecode = abi.encodePacked(
            type(ContrAccount).creationCode,
            abi.encode(
                owner,
```

```
                manager,
                positionManager,
                swapRouter,
                feeRecipient,
                feeRule
            )
        );

        bytes32 hash = keccak256(
            abi.encodePacked(bytes1(0xff), address(this), salt,
    keccak256(bytecode))
        );

        predicted = address(uint160(uint256(hash)));
    }
```

The salt itself is stable, because `_computeSalt()` only mixes `owner`, `block.chainid` and `VERSION`. The instability comes entirely from the constructor arguments that are hashed into the init code.

```
    function setManager(address m) external onlyAdmin {
        require(m != address(0), "ZERO_MANAGER");
        manager = m;
    }

    function setFeeRecipient(address r) external onlyAdmin {
        require(r != address(0), "ZERO_ADDR");
        feeRecipient = r;
    }

    function setFeeRule(address r) external onlyAdmin {
        require(r != address(0), "ZERO_FEE_RULE");
        feeRule = r;
    }
```

Since the admin can change those values before `createContrAccount()` is called, a previously predicted address can become stale while still looking authoritative to users or frontends that treat it as a prefunding target.

IMPACT:

A user can prefund a predicted account address that later becomes undeployable after an admin configuration change. Once the user creates the new account at the updated address, the assets sent to the stale predicted address can become permanently stranded, because no contract will ever be deployed at that address to recover them.

RECOMMENDATIONS:

Remove `predictContrAccount()` and do not expose its output as a prefunding target. If address preview must be retained for frontend convenience, bind the creation to the previewed value so a configuration change causes a revert instead of a silent divergence.

```
-    function predictContrAccount(address owner) external view returns
    (address predicted) {
-        bytes32 salt = _computeSalt(owner);
-        bytes memory bytecode = abi.encodePacked(
-            type(ContrAccount).creationCode,
-            abi.encode(owner, manager, positionManager, swapRouter,
    feeRecipient, feeRule)
-        );
-        bytes32 hash = keccak256(
-            abi.encodePacked(bytes1(0xff), address(this), salt,
    keccak256(bytecode))
-        );
-        predicted = address(uint160(uint256(hash)));
-    }

+    function createContrAccountExpecting(address expected)
+        external
+        payable
+        returns (address contr)
+    {
+        contr = _createContrAccount();
+        require(contr == expected, 'PREDICTION_STALE');
+    }
```

3.3 Spot-price accounting can inflate fee-share tiers for pool-priced deposits

SEVERITY: MEDIUM **STATUS:** Acknowledge

PATH:

`Manager.sol`, `ContrAccount.sol`

DESCRIPTION:

`depositToken()` permanently increases `matchedUsd` using `Manager.quoteTokenToUsd()`, while `_quoteTokenToUsd()` prices supported non-USD tokens from the current Uniswap V3 `slot0()` value. `slot0()` is an instantaneous pool price, not a TWAP or a trusted oracle, so a temporarily manipulated token/USD pool price can be written into long-lived fee-tier accounting.

```
    function _quoteTokenToUsd(address token, uint256 amount)
        internal
        view
        returns (uint256)
    {
        require(usdToken != address(0), "ZERO_USD_TOKEN");
        // ...
        address pool = tokenUsdPool[token];
        require(pool != address(0), "NO_USD_POOL");

        IUniswapV3PoolMinimal uniPool = IUniswapV3PoolMinimal(pool);

        address token0 = uniPool.token0();
        address token1 = uniPool.token1();

        require(
            (token0 == token && token1 == usdToken) ||
            (token0 == usdToken && token1 == token),
            "POOL_TOKEN_MISMATCH"
        );

        (uint160 sqrtPriceX96,,,,,,) = uniPool.slot0();
        require(sqrtPriceX96 > 0, "BAD_PRICE");

        uint256 usdRaw;
```

```
        if (token0 == token) {
            uint256 step1 = FullMath.mulDiv(amount,
    uint256(sqrtPriceX96), Q96);
            usdRaw = FullMath.mulDiv(step1, uint256(sqrtPriceX96), Q96);
        } else {
            uint256 step1 = FullMath.mulDiv(amount, Q96,
    uint256(sqrtPriceX96));
            usdRaw = FullMath.mulDiv(step1, Q96, uint256(sqrtPriceX96));
        }

        return _normalizeToUsdDecimals(usdRaw, usdTokenDecimals);
     }
```

The quoted value is written straight into permanent account state by the deposit path, and `_quoteNativeToUsd()` routes native deposits through the same spot-priced helper via the configured `weth` token.

```
        uint256 usdValue =
            IManagerQuote(manager).quoteTokenToUsd(token, amount);

        matchedUsd += usdValue;
```

Because fee sharing depends on `matchedUsd`, the manipulated accounting can qualify an account for a higher fee-share tier and reduce protocol fee revenue.

IMPACT:

A user can deposit while a configured token/USD pool spot price is temporarily inflated, causing `matchedUsd` to exceed the real deposited value and increasing later fee-share entitlement. Unlike a transient pricing error, the inflated value is persisted and keeps affecting every subsequent fee split until the position is closed.

RECOMMENDATIONS:

Do not use Uniswap V3 `slot0()` as a long term accounting oracle. Read an arithmetic mean tick over a configurable TWAP window instead, and require meaningful liquidity over that window.

```
+import '@uniswap/v3-periphery/contracts/libraries/OracleLibrary.sol';
+
```

```
 contract Manager {
+    uint32 public twapPeriod = 1800;
+
+    function setTwapPeriod(uint32 newTwapPeriod) external onlyAdmin {
+        require(newTwapPeriod >= 300, 'TWAP_TOO_SHORT');
+        twapPeriod = newTwapPeriod;
+    }

     function _quoteTokenToUsd(address token, uint256 amount)
         internal
         view
         returns (uint256)
     {
         ...
-        (uint160 sqrtPriceX96,,,,,,) = uniPool.slot0();
-        require(sqrtPriceX96 > 0, 'BAD_PRICE');
-
-        uint256 usdRaw;
-
-        if (token0 == token) {
-            uint256 step1 = FullMath.mulDiv(amount,
    uint256(sqrtPriceX96), Q96);
-            usdRaw = FullMath.mulDiv(step1, uint256(sqrtPriceX96), Q96);
-        } else {
-            uint256 step1 = FullMath.mulDiv(amount, Q96,
    uint256(sqrtPriceX96));
-            usdRaw = FullMath.mulDiv(step1, Q96, uint256(sqrtPriceX96));
-        }
+        (int24 arithmeticMeanTick, uint128 harmonicMeanLiquidity) =
+            OracleLibrary.consult(pool, twapPeriod);
+        require(harmonicMeanLiquidity > 0, 'NO_TWAP_LIQUIDITY');
+        require(amount <= type(uint128).max, 'AMOUNT_TOO_LARGE');
+
+        uint256 usdRaw = OracleLibrary.getQuoteAtTick(
+            arithmeticMeanTick,
+            uint128(amount),
+            token,
+            usdToken
+        );

         return _normalizeToUsdDecimals(usdRaw, usdTokenDecimals);
     }
 }
```

3.4 Silent balanceOf fallback can misaccount collected fees and principal

SEVERITY: MEDIUM **STATUS:** Fixed

PATH:

ContrAccount.sol

DESCRIPTION:

`_safeBalanceOf()` returned 0 when `balanceOf()` reverted or returned malformed data, instead of reverting.

```
    function _safeBalanceOf(address token, address account) internal view
        returns (uint256)
    {
        (bool ok, bytes memory data) =
            token.staticcall(
                abi.encodeWithSelector(IERC20.balanceOf.selector, account)
            );

        if (!ok || data.length < 32) {
            return 0;
        }

        return abi.decode(data, (uint256));
    }
```

That helper is used to compute the actual amount collected by balance delta in both `_collectAndSplitWithAmounts()` and `_collectPrincipalOnlyWithAmounts()`. If settlement succeeded while both balance reads were silently coerced to zero, the computed delta became zero, no protocol fee was transferred, and the collected tokens remained in the user-controlled account.

```
        uint256 bal0Before = _safeBalanceOf(token0, address(this));
        uint256 bal1Before = _safeBalanceOf(token1, address(this));

        INonfungiblePositionManager(positionManager).collect(
            INonfungiblePositionManager.CollectParams({
                tokenId: tokenId,
```

```
                recipient: address(this),
                amount0Max: type(uint128).max,
                amount1Max: type(uint128).max
            })
        );

        uint256 delta0 = _safeBalanceOf(token0, address(this)) -
    bal0Before;
        uint256 delta1 = _safeBalanceOf(token1, address(this)) -
    bal1Before;
```

IMPACT:

Collected tokens can remain in `ContrAccount`, and the owner can later withdraw them through owner withdrawal paths, causing protocol under-collection. Because the failure is silent, the settlement events still report a successful collection with zero protocol fee.

RECOMMENDATIONS:

Do not silently map failed `balanceOf()` calls to zero on settlement-critical paths; revert instead, so that a malformed token response aborts the whole settlement rather than under-reporting it.

```
 function _safeBalanceOf(address token, address account) internal view
    returns (uint256) {
     (bool ok, bytes memory data) =
         token.staticcall(
             abi.encodeWithSelector(IERC20.balanceOf.selector, account)
         );

-    if (!ok || data.length < 32) {
-        return 0;
-    }
+    require(ok, 'BALANCEOF_CALL_FAILED');
+    require(data.length >= 32, 'BALANCEOF_BAD_RETURN');

     return abi.decode(data, (uint256));
 }
```

3.5 positionOriginalUsd is recorded from desired mint amounts instead of actual deposited amounts

SEVERITY: MEDIUM **STATUS:** Acknowledge

PATH:

`Manager.sol`, `ContrAccount.sol`

DESCRIPTION:

`Manager.mintPosition()` and the zap-based position creation paths calculate `originalUsd` from the requested `amount0Desired` and `amount1Desired` before the Uniswap V3 mint is executed.

```
    function mintPosition(
        address contrAccount,
        MintParams calldata p,
        bool baseIsToken0
    )
        external
        onlyOperator
        returns (uint256 tokenId, uint128 liquidity, uint256 amount0,
    uint256 amount1)
    {
        uint256 originalUsd =
            _quoteTokenToUsd(p.token0, p.amount0Desired) +
            _quoteTokenToUsd(p.token1, p.amount1Desired);

        (tokenId, liquidity, amount0, amount1) =
            _mintPosition(contrAccount, p);

        IContrAccount(contrAccount).registerPosition(
            tokenId,
            originalUsd,
            baseIsToken0
        );
        // ...
    }
```

However, in Uniswap V3, `amount0Desired` and `amount1Desired` are maximum input amounts, not the actual amounts consumed by the minted position. The real values are returned by the mint call as `amount0` and `amount1`, and they are already decoded and emitted in `PositionCreateSnapshot`, but they are not used for accounting. The same pattern is repeated in `zapCreatePosition()`, `zapCreatePositionETH()`, `zapCreatePositionV2()` and `zapCreatePositionETHV2()`.

`ContrAccount.closePosition()` later decreases `matchedUsd` using the stored `positionOriginalUsd[tokenId]`, so the unwind path also depends on this inflated value.

```
        if (isOwnerInitiated) {
            uint256 originalUsd = positionOriginalUsd[tokenId];
            if (originalUsd > 0) {
                uint256 newMatchedUsd;

                if (originalUsd >= matchedUsd) {
                    matchedUsd = 0;
                    newMatchedUsd = 0;
                } else {
                    matchedUsd -= originalUsd;
                    newMatchedUsd = matchedUsd;
                }
                // ...
            }
        }
```

IMPACT:

The account records and later unwinds a USD value based on intended mint inputs rather than the real principal actually deployed into the Uniswap V3 position. Because the close path subtracts the same overstated number, a single position close can zero out or over-reduce `matchedUsd` that was legitimately contributed by other deposits, producing inconsistent fee tiering in both directions.

RECOMMENDATIONS:

Modify `Manager.mintPosition()` and all zap-based position creation functions so that `originalUsd` is calculated after `_mintPosition()` returns, using the actual `amount0` and `amount1` values returned by Uniswap V3.

```
    {
```

```
-         uint256 originalUsd =
-             _quoteTokenToUsd(p.token0, p.amount0Desired) +
-             _quoteTokenToUsd(p.token1, p.amount1Desired);
-
          (tokenId, liquidity, amount0, amount1) =
              _mintPosition(contrAccount, p);

+         uint256 originalUsd =
+             _quoteTokenToUsd(p.token0, amount0) +
+             _quoteTokenToUsd(p.token1, amount1);
+
          IContrAccount(contrAccount).registerPosition(
              tokenId,
              originalUsd,
              baseIsToken0
          );
```

3.6 Stale operator privileges survive admin rotation and preserve execution control

SEVERITY: LOW **STATUS:** Fixed

PATH:

`Manager.sol`

DESCRIPTION:

The constructor grants the initial admin full operator execution rights through `isOperator[_admin] = true` and `_addOperator(_admin)`, but `setAdmin()` only updated the `admin` storage slot.

```
    constructor(
        address _admin,
        address _positionManager,
        address _swapRouter
    ) {
        ...
        admin = _admin;
        positionManager = _positionManager;
        swapRouter = _swapRouter;

        isOperator[_admin] = true;
        _addOperator(_admin);
        _setDefaultApprovalResetTokens();

        emit OperatorUpdated(_admin, true);
    }

    function setAdmin(address newAdmin) external onlyAdmin {
        require(newAdmin != address(0), "ZERO_ADMIN");
        emit AdminUpdated(admin, newAdmin);
        admin = newAdmin;
    }
```

Sensitive portfolio actions such as `mintPosition`, `swapExactInputSingle` and `closePosition` continue to trust the `onlyOperator` role, so a stale operator mapping

preserves execution control after admin rotation.

IMPACT:

Any governance rotation that executes `setAdmin()` without an immediate matching `setOperator()` update leaves the former admin with a live `onlyOperator` privilege set, and the new admin without one until a separate transaction is sent.

RECOMMENDATIONS:

Revoke the old admin operator rights and grant the new admin the same execution role atomically inside `setAdmin()`, so that authority transfer cannot leave a window with two effective operators.

```
 function setAdmin(address newAdmin) external onlyAdmin {
     require(newAdmin != address(0), 'ZERO_ADMIN');
+    address oldAdmin = admin;
+
+    if (oldAdmin != newAdmin) {
+        isOperator[oldAdmin] = false;
+        _removeOperator(oldAdmin);
+
+        isOperator[newAdmin] = true;
+        _addOperator(newAdmin);
+
+        emit OperatorUpdated(oldAdmin, false);
+        emit OperatorUpdated(newAdmin, true);
+    }
+
-    emit AdminUpdated(admin, newAdmin);
+    emit AdminUpdated(oldAdmin, newAdmin);
     admin = newAdmin;
 }
```

3.7 Nominal deposit amounts can overstate fee-tier accounting for supported non-standard tokens

SEVERITY: LOW **STATUS:** Acknowledge

PATH:

ContrAccount.sol,Manager.sol

DESCRIPTION:

depositToken() credits matchedUsd from the user-supplied amount after calling quoteTokenToUsd(token, amount), but never checks how many tokens were actually received by the account. If any supported token is fee-on-transfer, taxed, rebasing, or otherwise transfers less than amount, matchedUsd is credited against the nominal input instead of the actual received balance.

```
    function depositToken(address token, uint256 amount)
        external
        onlyOwner
        nonReentrant
    {
        require(amount > 0, "INVALID_AMOUNT");

        (bool ok, bytes memory ret) =
            token.call(
                abi.encodeWithSelector(
                    TRANSFER_FROM_SELECTOR,
                    msg.sender,
                    address(this),
                    amount
                )
            );

        require(ok, "TRANSFER_FROM_CALL_FAILED");

        if (ret.length > 0) {
            require(abi.decode(ret, (bool)), "TRANSFER_FROM_FAILED");
        }
```

```
        emit DepositToken(owner, address(this), token, amount);

        uint256 usdValue =
            IManagerQuote(manager).quoteTokenToUsd(token, amount);

        matchedUsd += usdValue;
        // ...
    }
```

IMPACT:

For supported non-standard tokens, `matchedUsd` can become larger than the account's real received value and overstate later fee-tier eligibility. The emitted `DepositToken` and `MatchedUsdIncreased` events also report the nominal amount, so off-chain indexers inherit the same discrepancy.

RECOMMENDATIONS:

Restrict the supported token set to standard ERC-20 implementations that transfer the exact requested amount. If non-standard tokens must be supported, credit the balance delta actually received instead of the requested amount.

```
     function depositToken(address token, uint256 amount) external
   onlyOwner nonReentrant {
        require(amount > 0, "INVALID_AMOUNT");
+       uint256 balBefore = _safeBalanceOf(token, address(this));

        // ... transferFrom(msg.sender, address(this), amount) ...

-       emit DepositToken(owner, address(this), token, amount);
+       uint256 received = _safeBalanceOf(token, address(this)) -
   balBefore;
+       require(received > 0, "NO_TOKENS_RECEIVED");
+
+       emit DepositToken(owner, address(this), token, received);

-       uint256 usdValue = IManagerQuote(manager).quoteTokenToUsd(token,
   amount);
+       uint256 usdValue = IManagerQuote(manager).quoteTokenToUsd(token,
   received);
```

```
            matchedUsd += usdValue;
        }
```

3.8 Unbounded fee-share bps can block fee collection and position closure

SEVERITY: LOW **STATUS:** Acknowledge

PATH:

`ContrAccount.sol`

DESCRIPTION:

`_collectAndSplitWithAmounts()` trusts `feeRule.getUserFeeBps(matchedUsd)` as a valid basis-point percentage, but `ContrAccount` never enforces `bps <= 10000`.

```
        uint16 bps = IFeeRule(feeRule).getUserFeeBps(matchedUsd);

        feeAmount0ToOwner = (delta0 * bps) / 10000;
        feeAmount1ToOwner = (delta1 * bps) / 10000;

        protocolFee0 = delta0 - feeAmount0ToOwner;
        protocolFee1 = delta1 - feeAmount1ToOwner;
```

If `bps` exceeds 10000, `feeAmount0ToOwner` becomes larger than `delta0` and the subtraction reverts on underflow. Both `collectFees()` and `closePosition()` depend on this same split path before completing normal settlement, and `ContrAccount` binds `feeRule` at construction with no revalidation of the value it returns.

The currently deployed `FeeRule.sol` bounds the configured value in `setRule()`, but that guard lives in a separate contract and does not protect an account that is constructed against a different rule implementation.

```
    function setRule(uint256 threshold, uint16 userFeeBps) external
   onlyAdmin {
        require(userFeeBps <= 10000, "INVALID_BPS");
        // ...
    }
```

IMPACT:

A bad `feeRule` return value can make fee collection and position closure revert for active positions, blocking normal fee settlement and exit until the rule is corrected. Because the account has no way to change its `feeRule` after construction, affected accounts would have no in-contract recovery path.

RECOMMENDATIONS:

Clamp or validate the returned basis points inside `ContrAccount` before using them in fee split calculations, so that settlement remains available regardless of the rule contract behaviour.

```
 uint16 bps = IFeeRule(feeRule).getUserFeeBps(matchedUsd);
+if (bps > 10000) {
+    bps = 10000;
+}
 feeAmount0ToOwner = (delta0 * bps) / 10000;
 feeAmount1ToOwner = (delta1 * bps) / 10000;
```

3.9 Manager external updates can make existing ContrAccount instances incompatible with managed operations

SEVERITY: LOW **STATUS:** Fixed

PATH:

`Manager.sol`, `ContrAccount.sol`

DESCRIPTION:

`Manager` allowed the admin to update the active Uniswap V3 `positionManager` and `swapRouter`, while each `ContrAccount` stores its own copies of those addresses at construction time.

```
    function setExternals(address pm, address router) external onlyAdmin {
        require(pm != address(0), 'ZERO_PM');
        require(router != address(0), 'ZERO_ROUTER');
        positionManager = pm;
        swapRouter = router;
    }

    constructor(
        address _owner,
        address _manager,
        address _positionManager,
        address _swapRouter,
        ...
    ) {
        ...
        positionManager = _positionManager;
        swapRouter = _swapRouter;
    }
```

The account side enforces its stored copies on every managed action, so once the two sides diverge the account rejects the manager's calls.

```
    function _isAllowedSpender(address spender) internal view returns
    (bool) {
        return (
```

```
            spender == positionManager ||
            spender == swapRouter ||
            spender == PERMIT2
        );
    }

    function execute(address target, uint256 value, bytes calldata data)
        external
        onlyManager
        nonReentrant
        returns (bytes memory)
    {
        require(
            target == swapRouter || target == positionManager,
            "INVALID_TARGET"
        );
        ...
    }
```

IMPACT:

Existing accounts can become incompatible with managed operations after an external address update. Operator-driven flows may revert with `INVALID_TARGET` or `INVALID_SPENDER` when executed against old accounts, and there is no account-side setter to resynchronize them.

RECOMMENDATIONS:

Use one consistent upgrade model across `Manager` and `ContrAccount`. Either keep external dependencies immutable per account, or add an explicit synchronized migration path that updates account-side trusted endpoints before the manager switches globals. The project adopted the immutable model, disabling the global setters on both `Manager` and `Factory`.

```
 function setExternals(address pm, address router) external onlyAdmin {
     require(pm != address(0), "ZERO_PM");
     require(router != address(0), "ZERO_ROUTER");
-    positionManager = pm;
-    swapRouter = router;
+
+    revert("EXTERNALS_IMMUTABLE");
 }
```

3.10 ContrAccount admin is stored but never used

SEVERITY: INFO **STATUS:** Fixed

PATH:

`ContrAccount.sol`

DESCRIPTION:

`ContrAccount` received an `_admin` argument during construction and stored it in the `admin` state variable, but no later execution path used that value for authorization or configuration. All access control in the account is expressed through `onlyOwner`, `onlyManager` and `onlyOwnerOrManager`.

```
    constructor(
        address _owner,
        address _manager,
        address _admin,
        address _positionManager,
        address _swapRouter,
        address _feeRecipient,
        address _feeRule
    ) {
        ...
        admin = _admin;
        ...
    }
```

IMPACT:

This unused state variable can confuse integrators and reviewers by implying that the protocol admin has some direct authority over `ContrAccount` instances when no such authority is implemented. It also increases deployment and storage complexity without providing functional benefit, and it adds a mutable factory input to the CREATE2 init code that changes every predicted account address.

RECOMMENDATIONS:

Remove the unused `admin` constructor argument and state variable from `ContrAccount`, and update `Factory.createContrAccount()` and `Factory.predictContrAccount()` to the reduced argument list.

```
    constructor(
        address _owner,
        address _manager,
-       address _admin,
        address _positionManager,
        address _swapRouter,
        address _feeRecipient,
        address _feeRule
    ) {
        ...
-       admin = _admin;
        ...
    }
```

3.11 zapCreatePositionETH V1 does not ensure swap.amountIn is bounded by ethAmount

SEVERITY: INFO **STATUS:** Acknowledge

PATH:

Manager.sol

DESCRIPTION:

`Manager.zapCreatePositionETH()` wraps `z.ethAmount` into WETH before optionally executing a swap, but the V1 ETH zap path does not check that `z.swap.amountIn <= z.ethAmount`.

```
    function zapCreatePositionETH(
        address contrAccount,
        ZapETHParams calldata z,
        bool baseIsToken0
    )
        external
        onlyOperator
    {
        require(contrAccount != address(0), 'ZERO_CONTR');
        require(z.weth != address(0), 'ZERO_WETH');
        require(z.ethAmount > 0, 'ZERO_ETH');

        _validateWeth(z.weth);
        _validateZapSingle(z.swap, z.mint);

        IContrAccount(contrAccount).wrapETH(z.weth, z.ethAmount);
        // ...
        if (z.swap.amountIn > 0) {
            require(z.swap.tokenIn == z.weth, 'NATIVE_SWAP_IN_NOT_WETH');

            didSwap = true;
            _swapExactInputSingle(contrAccount, z.swap);
        }
        // ...
    }
```

The V2 ETH zap path performs exactly this validation through `_validateZapV2ETH()`, so the two variants are inconsistent.

```
    function _validateZapV2ETH(
        uint256 ethAmount,
        SwapParams calldata s0,
        SwapParams calldata s1
    ) internal pure {
        uint256 totalSwap = s0.amountIn + s1.amountIn;

        if (totalSwap > ethAmount) {
            revert ETH_SWAP_EXCEEDS_INPUT();
        }
    }
```

As a result, the V1 ETH zap path can consume WETH that was already present in the `ContrAccount`, not only the WETH produced by the current `ethAmount`.

IMPACT:

This does not create a new direct asset-transfer capability because the caller must be an authorized operator, but it makes the V1 ETH zap semantics less precise than expected: a zap request that appears to be funded by `ethAmount` may also consume pre-existing WETH held by the account.

RECOMMENDATIONS:

Apply the same input bound that the V2 path already enforces, so that a V1 ETH zap can only spend the WETH it just produced.

```
+    function _validateZapETH(
+        uint256 ethAmount,
+        SwapParams calldata s
+    ) internal pure {
+        if (s.amountIn > ethAmount) {
+            revert ETH_SWAP_EXCEEDS_INPUT();
+        }
+    }

     function zapCreatePositionETH(
         address contrAccount,
         ZapETHParams calldata z,
```

```
          bool baseIsToken0
      )
          external
          onlyOperator
      {
          ...
          _validateWeth(z.weth);
          _validateZapSingle(z.swap, z.mint);
+         _validateZapETH(z.ethAmount, z.swap);

          IContrAccount(contrAccount).wrapETH(z.weth, z.ethAmount);
          ...
      }
```

4. CONCLUSION

In this audit, we thoroughly analyzed **Bullion Pool** smart contract implementation. The problems found are described and explained in detail in Section 3. The problems found in the audit have been communicated to the project leader. We therefore consider the audit result to be **PASSED**.

To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.

5. APPENDIX

5.1 Basic Coding Assessment

5.1.1 Apply Verification Control

Description	The security of apply verification
Result	Not found
Severity	CRITICAL

5.1.2 Authorization Access Control

Description	Permission checks for external integral functions
Result	Not found
Severity	CRITICAL

5.1.3 Forged Transfer Vulnerability

Description	Assess whether there is a forged transfer notification vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.4 Transaction Rollback Attack

Description	Assess whether there is transaction rollback attack vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.5 Transaction Block Stuffing Attack

Description	Assess whether there is transaction blocking attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.6 Soft Fail Attack Assessment

Description	Assess whether there is soft fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.7 Hard Fail Attack Assessment

Description	Examine for hard fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.8 Abnormal Memo Assessment

Description	Assess whether there is abnormal memo vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.9 Abnormal Resource Consumption

Description	Examine whether abnormal resource consumption in contract processing
Result	Not found
Severity	CRITICAL

5.1.10 Random Number Security

Description	Examine whether the code uses insecure random number
Result	Not found
Severity	CRITICAL

5.2 Advanced Code Scrutiny

5.2.1 Cryptography Security

Description	Examine for weakness in cryptograph implementation
Result	Not found
Severity	HIGH

5.2.2 Account Permission Control

Description	Examine permission control issue in the contract
Result	Not found
Severity	MEDIUM

5.2.3 Malicious Code Behavior

Description	Examine whether sensitive behavior present in the code
Result	Not found
Severity	MEDIUM

5.2.4 Sensitive Information Disclosure

Description	Examine whether sensitive information disclosure issue present in the code
Result	Not found
Severity	MEDIUM

5.2.5 System API

Description	Examine whether system API application issue present in the code
Result	Not found
Severity	LOW

6. DISCLAIMER

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to the Company in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes without ExVul's prior written consent.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts ExVul to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. ExVul's position is that each company and individual are responsible for their own due diligence and continuous security. ExVul's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

7. REFERENCES

[1] MITRE. CWE-191: Integer Underflow (Wrap or Wraparound). https://cwe.mitre.org/data/definitions/191.html.

[2] MITRE. CWE-197: Numeric Truncation Error. https://cwe.mitre.org/data/definitions/197.html.

[3] MITRE. CWE-400: Uncontrolled Resource Consumption. https://cwe.mitre.org/data/definitions/400.html.

[4] MITRE. CWE-440: Expected Behavior Violation. https://cwe.mitre.org/data/definitions/440.html.

[5] MITRE. CWE-684: Protection Mechanism Failure. https://cwe.mitre.org/data/definitions/693.html.

[6] MITRE. CWE CATEGORY: 7PK - Security Features. https://cwe.mitre.org/data/definitions/254.html.

[7] MITRE. CWE CATEGORY: Behavioral Problems. https://cwe.mitre.org/data/definitions/438.html.

[8] MITRE. CWE CATEGORY: Numeric Errors. https://cwe.mitre.org/data/definitions/189.html.

[9] MITRE. CWE CATEGORY: Resource Management Errors. https://cwe.mitre.org/data/definitions/399.html.

[10] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology

8. About Exvul Security

Premier Security for the Web3 Ecosystem

ExVul is a premier Web3 security firm committed to forging a secure and trustworthy decentralized ecosystem. Our elite team consists of security veterans from world-leading technology and blockchain security firms, including Huawei, YBB Captical, Qihoo 360, Amber, ByteDance, MoveBit, and PeckShield. Team member Nolan is ranked as a top-40 whitehat on Immunefi and is the platform's sole All-Star in the APAC region.

Our expertise covers the full spectrum of Web3 security. We conduct **meticulous smart contract audits**, having fortified thousands of projects on chains like Evm, Solana, Aptos, Sui etc. Our **Blockchain Protocol Audits** secure the core infrastructure of L1/L2 by uncovering deep-seated vulnerabilities. We also offer **comprehensive wallet audits** to protect user assets and provide **proactive web3 pentest**, enabling partners to neutralize threats before they strike.

Trusted by industry leaders, ExVul is the security partner for **OKX, Bitget, Cobo, Infini, Stacks, Aptos, Sui, CoreDAO, Sei** etc.

EXVUL

Contact

Website
www.exvul.com

Email
contact@exvul.com

Twitter
@EXVULSEC

Github
github.com/EXVUL-Sec

ExVul